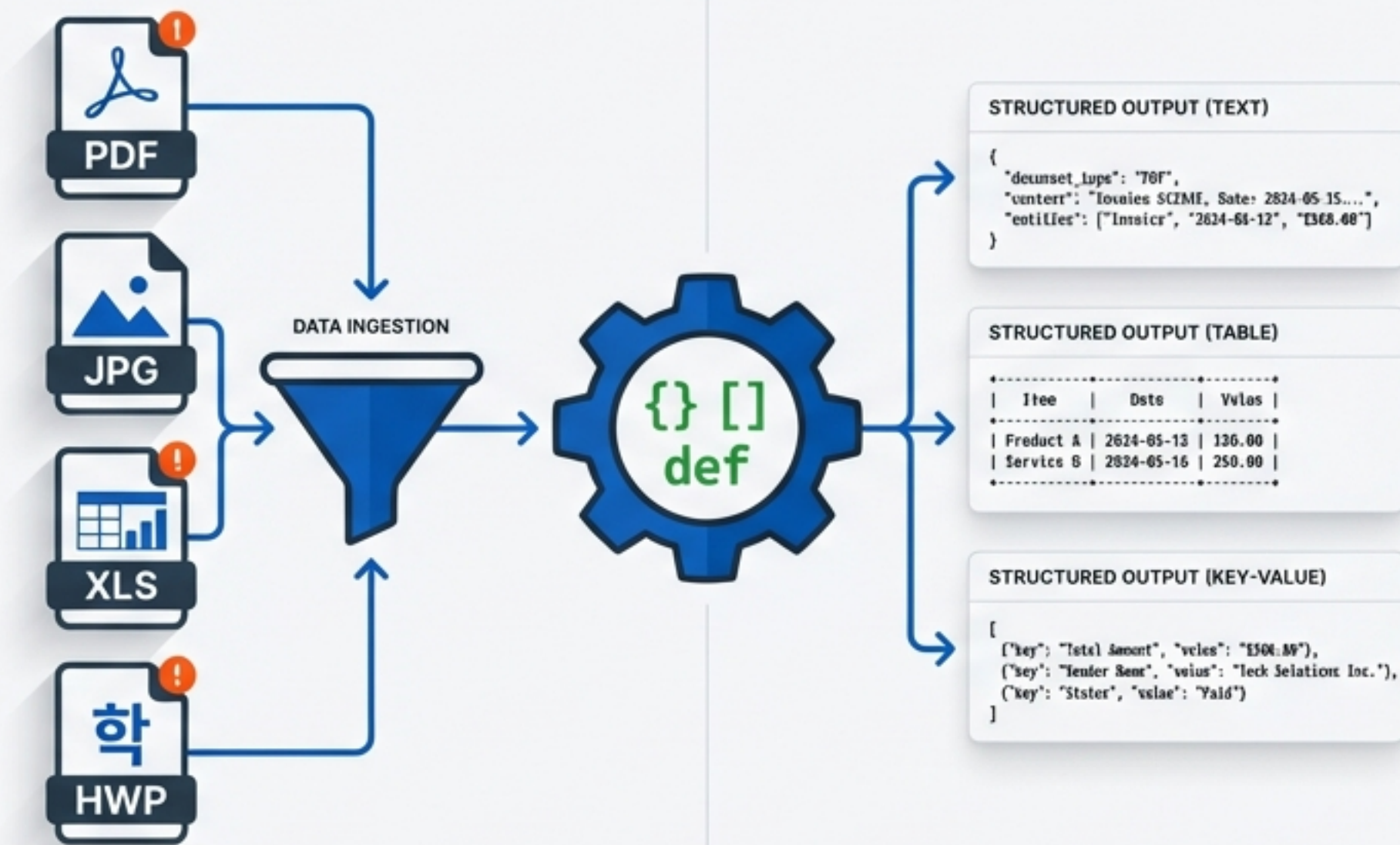


파이썬 자동화 레시피: 기초 문법부터 텍스트 추출까지

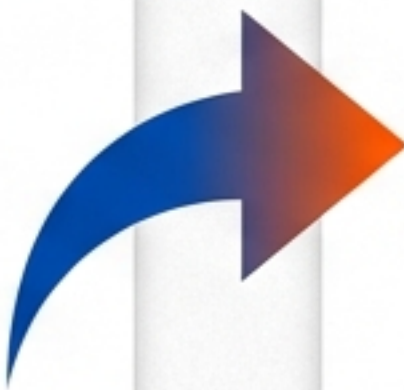
Part 1. 핵심 기초 문법 &
Part 2. 실전 파일 데이터 추출
(PDF, 이미지, Office, HWP)



이 튜토리얼의 로드맵

Part 1 - 기초 툴킷 (The Foundation)

- ✓ 변수와 데이터 타입
(Variables & Types)
- ✓ 입출력과 제어문
(Input/Output & Control Flow)
- ✓ 데이터 구조와 문자열
(Lists, Dicts, Strings)
- ✓ 예외 처리와 파일 입출력
(Exceptions & File I/O)



Part 2 - 실전 텍스트 추출 (The Application)

- PDF: `PyMuPDF` (텍스트 기반 문서)
- Image (OCR): `pytesseract`
(스캔 문서)
- MS Office: `win32com`
(Excel, Outlook 자동화)
- Hancom: `pyhwp`
(아래아한글 자동화)

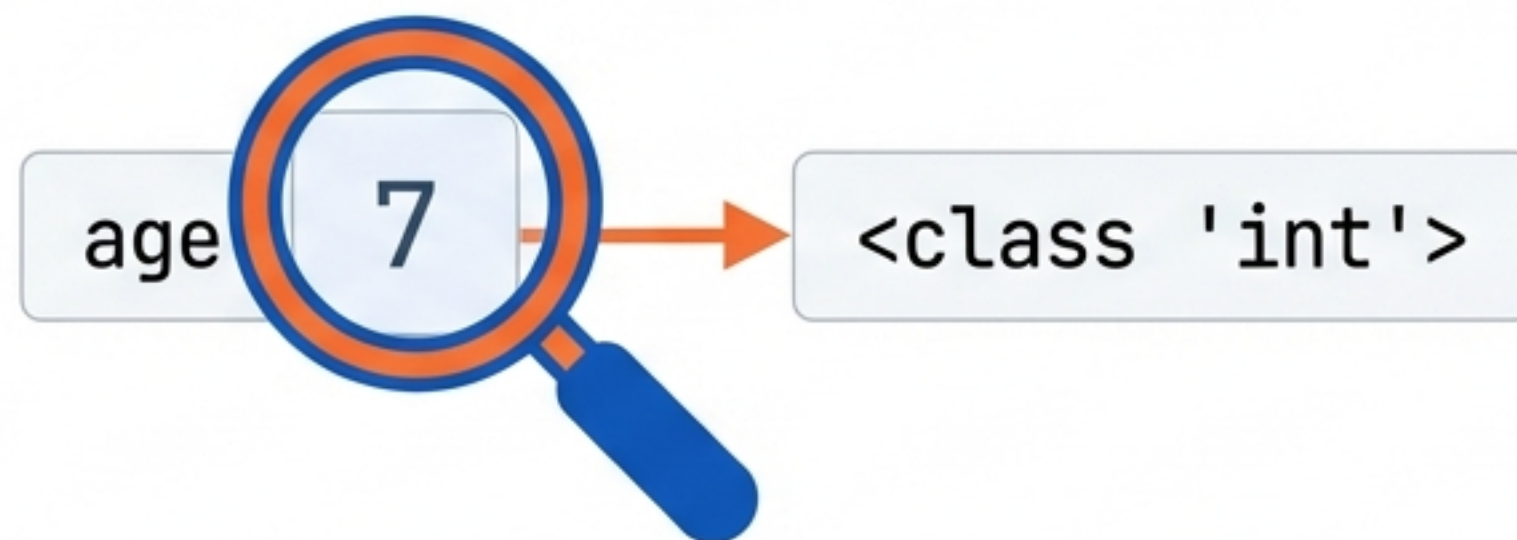
변수와 데이터 타입 (Variables & Data Types)

변수는 값을 담는 상자이며, 파이썬은 데이터 타입을 동적으로 결정합니다.

```
name = "Leo"    # 문자열 (String)
age = 7          # 정수 (Integer)
height = 5.6    # 실수 (Float)
is_cat = True    # 불린 (Boolean)
```

```
# 타입 확인
print(type(age))
```

Type Investigation



`type()` 함수를 사용하여 변수의 데이터 타입을 동적으로 확인할 수 있습니다.

입출력과 형 변환 (Input/Output)

`input()` 함수는 사용자의 입력을 항상 **문자열(String)**로 받습니다.
숫자 계산을 위해서는 `int()`나 `float()` 변환이 필요합니다.

```
# 입력 받기 (문자열)
age_str = input("나이를 입력하세요: ")

# 정수로 변환
age = int(age_str)
print(f"내년에는 {age + 1}살이 됩니다.")
```

```
나이를 입력하세요: 20
내년에는 21살이 됩니다.
```

출력 포매팅 옵션 (Print Options)

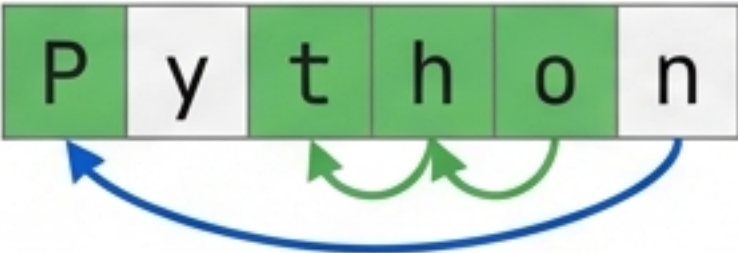
```
print('A', 'B', sep='-') → A-B
print('A', end=' ') → 줄바꿈 없음
```


데이터 구조와 문자열 처리

리스트 슬라이싱과 필수 문자열 함수를 통해 데이터를 효율적으로 처리합니다.

리스트 슬라이싱 (List Slicing)

```
text = "Python"
```



text[::2] → "Pto"

text[::-1] → "nohtyP"

문자열 필수 함수 (Essential Methods)

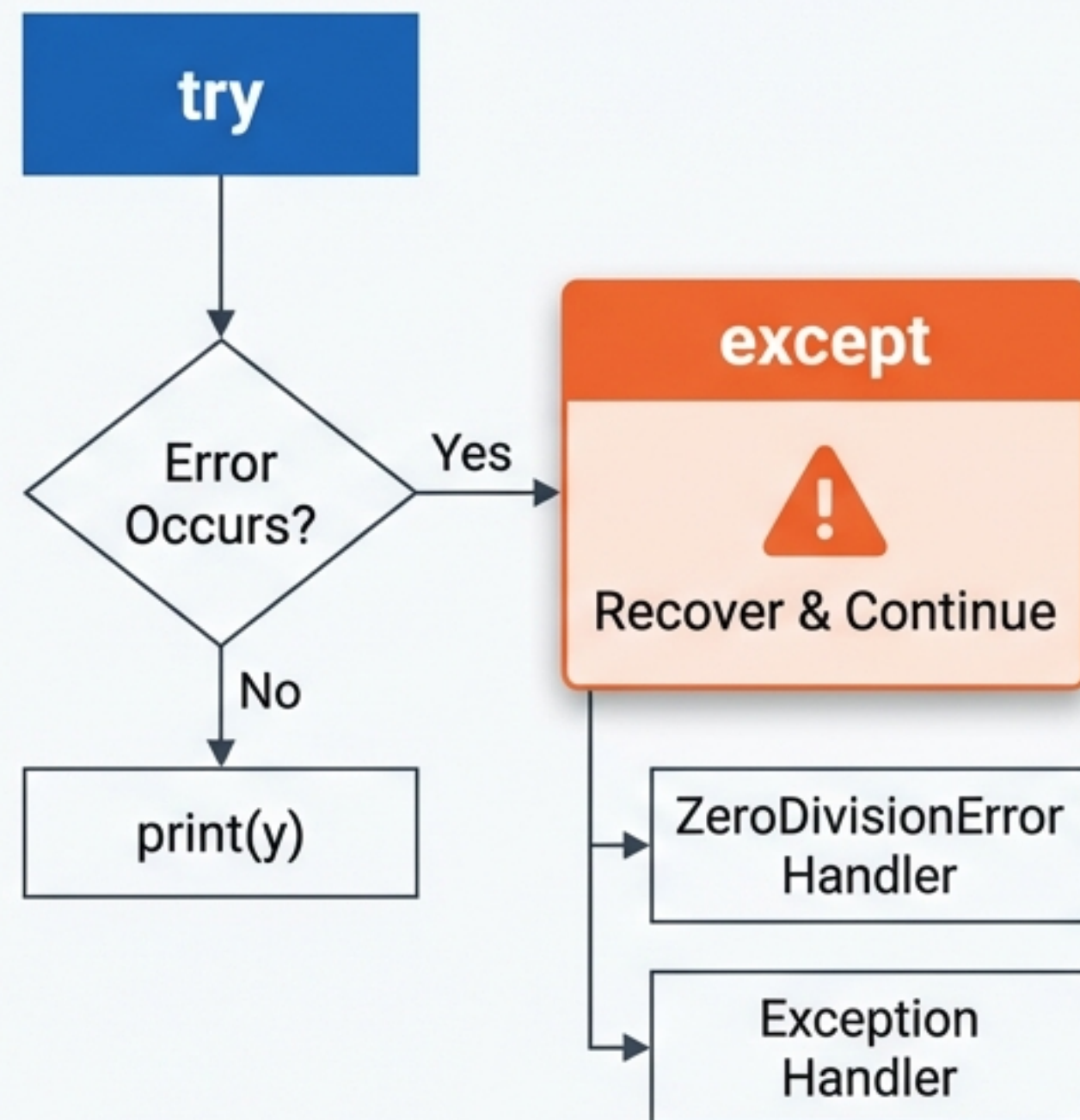
	Code	Description
1	.strip()	앞뒤 공백 제거
2	.replace('A', 'B')	문자열 치환
3	.split()	공백 기준 리스트 분리

```
user = {"name": "Leo",
        "age": 42
}
```


예외 처리 (Exception Handling)

에러가 발생해도 프로그램이 멈추지 않도록 안전장치를 만듭니다.

```
try:
    x = int(input("나눌 숫자: "))
    y = 10 / x
    print(y)
except ZeroDivisionError as e:
    print("0으로 나눌 수 없습니다.", e)
except Exception as e:
    print("알 수 없는 에러가 발생했습니다.", e)
```



파일 입출력 (File I/O)

`with open(...)` 구문을 사용하면 파일을 다룬 후 자동으로 닫아주어 안전합니다.

```
with open(...) as file:
```

File Operations (read/write)

Auto-Close
(Resource Released)



파일 읽기 (Read)

```
with open('sample.txt', 'r', encoding='utf-8') as file:  
    content = file.read()  
    print(content)
```

파일 쓰기 (Write)

```
with open('output.txt', 'w', encoding='utf-8') as file:  
    file.write("파이썬 파일 쓰기 테스트")
```


Part 2. 실전 텍스트 추출

다양한 파일 형식에 맞춰 최적의 라이브러리를 선택해야 합니다.



PyMuPDF (fitz)

빠르고 강력한 PDF 텍스트 추출



pytesseract

이미지 속 글자 인식 (OCR)



pywin32 (win32com)

MS Office 앱 직접 제어



pyhwp

한글(HWP) 문서 자동화

설치 명령어: `pip install pymupdf pytesseract pywin32 pyhwp`

PDF 텍스트 추출 (PyMuPDF)

Library: `import fitz`

1 문서 열기:

```
doc = fitz.open('file.pdf') ↪
```



2 페이지 순회:

```
for page in doc: ↪
```



3 텍스트 추출:

```
text = page.get_text() ↪
```

구현 예시 (Implementation)

```
import fitz

doc = fitz.open('sample.pdf')

def to_plain_text(text):
    # 불필요한 줄바꿈 제거
    return text.replace('\n', '')

for page in doc:
    raw_text = page.get_text()
    clean_text = to_plain_text(raw_text)
    print(clean_text)
```

Note: PyPDF2보다 추출 정확도가 높고 속도가 빠릅니다.

이미지 텍스트 추출 (OCR)

Library: pytesseract, PIL

⚠ Prerequisite: Tesseract-OCR 엔진(실행 파일)이 PC에 별도로 설치되어 있어야 합니다.

구현 예시 (Implementation)

Setup:

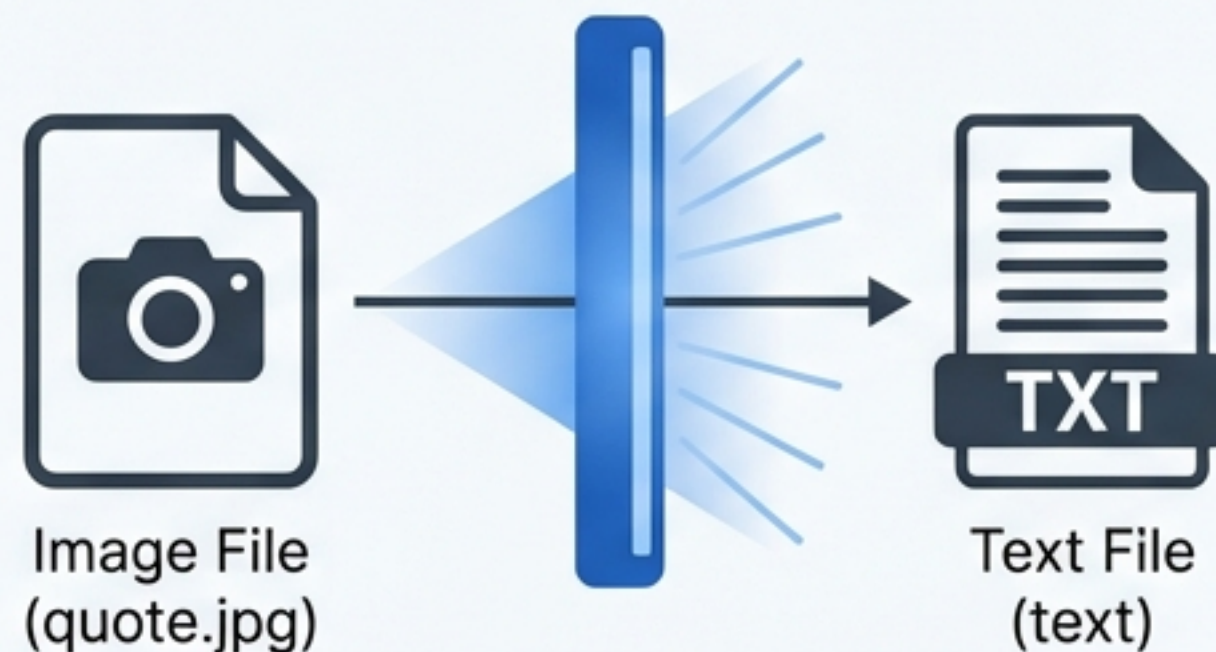
```
import pytesseract
from PIL import Image

# 윈도우 사용자는 실행 파일 경로 지정 필수
pytesseract.pytesseract.tesseract_cmd = r'C:\Path\To\tesseract.exe'
```

Execution:

```
img = Image.open('quote.jpg')

# lang='kor+eng' 옵션으로 한글과 영어를 동시에 인식
text = pytesseract.image_to_string(img, lang='kor+eng')
print(text)
```



스캔된 PDF에서 텍스트 추출하기

Library: PyMuPDF, pytesseract, PIL

텍스트 레이어가 없는 이미지는 **PDF → 이미지 → OCR** 과정을 거쳐야 합니다.



구현 예시 (Implementation)

```
1 import fitz
2 from PIL import Image
3 import pytesseract
4
5 doc = fitz.open('scan.pdf')
6 page = doc[0]
7
8 # 1. PDF 페이지를 이미지로 변환 (해상도 4배 확대하여 인식을 향상)
9 pix = page.get_pixmap(matrix=fitz.Matrix(4, 4))
10 img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
11
12 # 2. 이미지에서 텍스트 추출
13 text = pytesseract.image_to_string(img, lang='kor')
14 print(text)
```


엑셀(Excel)과 아웃룩(Outlook) 자동화

Library: win32com.client

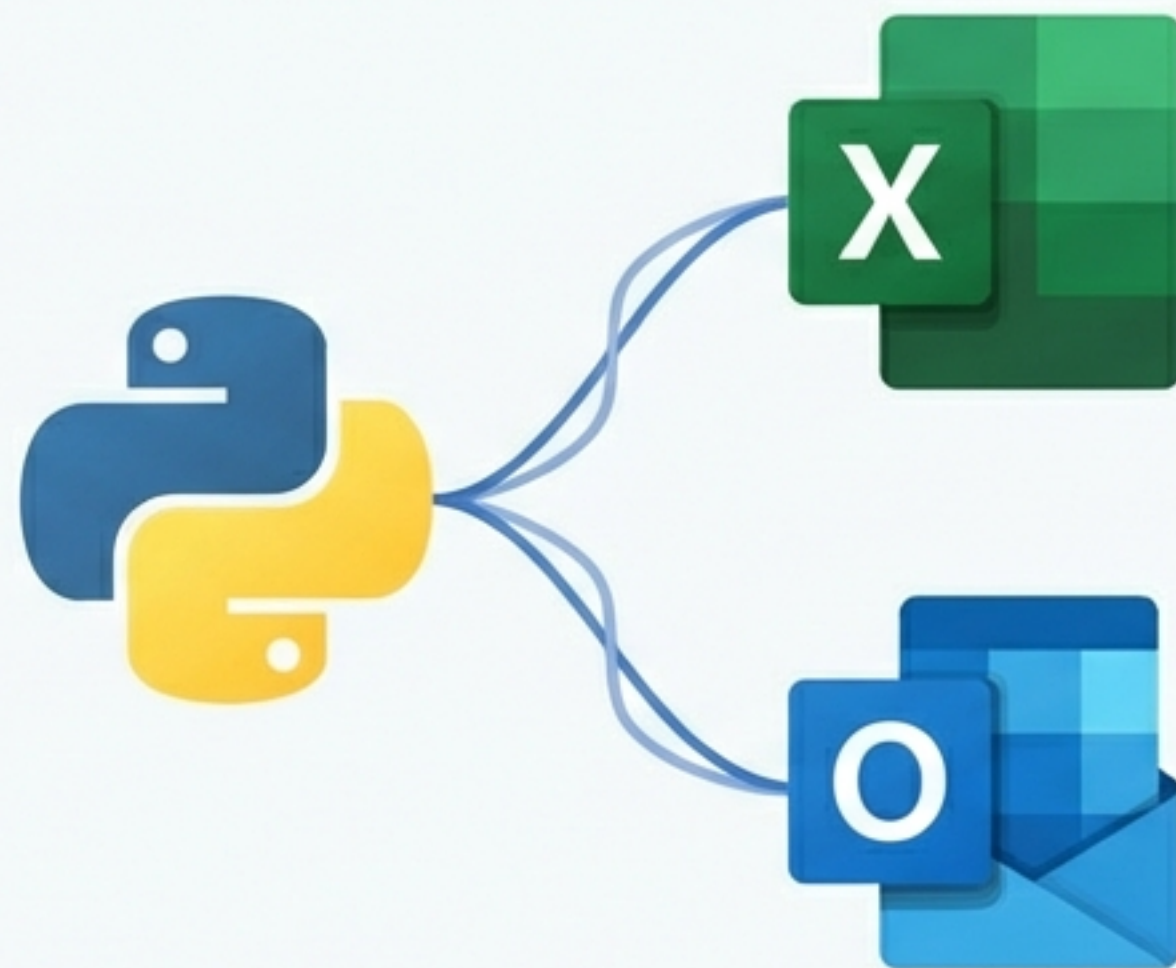
엑셀이나 아웃룩 프로그램을 직접 실행시켜 제어하는 '봇(Bot)' 방식입니다.

구현 예시 (Implementation)

```
import win32com.client as win32

# 1. 엑셀 실행 및 데이터 복사
excel = win32.gencache.EnsureDispatch('Excel.Application')
wb = excel.Workbooks.Open(r'C:\Data\Book1.xlsx')
wb.Sheets(1).Range("A1:B3").Copy()

# 2. 아웃룩 실행 및 붙여넣기
outlook = win32.Dispatch("Outlook.Application")
msg = outlook.CreateItem(0)
msg.Display() # 화면에 창 띄우기
msg.GetInspector.WordEditor.Range(0, 0).Paste() # 붙여넣기
```



한컴오피스(HWP) 자동화

Library: pyhwp

윈도우 전용 라이브러리로, 복잡한 HWP 제어를 단순화해줍니다.

구현 예시 (Implementation)

```
1 from pyhwp import Hwp
2
3 hwp = Hwp() # 한/글 프로그램 실행
4 hwp.open("report.hwp") # 파일 열기
5
6 # 텍스트 삽입 및 줄바꿈
8 hwp.insert_text("자동화된 텍스트입니다.")
9 hwp.BreakPara()
10
11 hwp.Quit() # 종료
```



요약: 상황별 도구 선택 가이드

입력 데이터 (Input)	추천 도구 (Tool)	특징 (Key Feature)
텍스트형 PDF	PyMuPDF (fitz)	빠름, 텍스트 레이어 직접 접근
이미지 / 스캔 PDF	pytesseract	OCR 엔진 필요, 해상도 중요
MS Excel / Outlook	win32com	실제 앱 구동 , 윈도우 전용
HWP (한글)	pyhwp	한글 오토메이션 매크로 제어

Tip: 모든 파일 작업 후에는 반드시 `.close()` 또는 `Quit()`으로 리소스를 해제하세요.

더 공부하기 & 참고 자료



Python Basics: Real Python Tutorials & Cheatsheets



PDF Processing: PyMuPDF Documentation (readthedocs.io)



OCR Engine: Tesseract-OCR GitHub Repository



Windows Automation: pywin32 documentation



HWP Automation: pyhwp Documentmentation

반복적인 업무는 파이썬에게 맡기고,
더 중요한 문제 해결에 집중하세요.